Claude Code + Quarry: A Z Specification

Layer 2 — Knowledge Capture State

March 2026

Contents

1 Introduction **3**

2 Base Types (from claude-code.tex) **3**

3 Quarry Types **3**

4 Global Constants **3**

5 State **4**

6 Initialization **4**

7 Quarry Frame Condition **5**

8 Session Lifecycle Operations **5**
- 8.1 StartSessionWithSync . . . 5
- 8.2 StartSessionNoSync . . . 6
- 8.3 SubmitPrompt . . . 6

9 Tool Call Operations **7**
- 9.1 BeginToolCall . . . 7
- 9.2 PreToolHookAllow . . . 7
- 9.3 PreToolHookDeny . . . 8
- 9.4 GrantPermission . . . 8
- 9.5 DenyPermission . . . 9
- 9.6 CompleteToolSuccess . . . 9
- 9.7 CompleteToolFailure . . . 9

10 Quarry Operations **10**
- 10.1 CaptureWebFetch . . . 10
- 10.2 SwitchDatabase . . . 10
- 10.3 SearchKnowledge . . . 11
- 10.4 InjectKnowledgeHint . . . 11

11 Subagent Operations **12**

12 Response and Stop **13**

13 Context Management — Compaction Safety **14**
- 13.1 CaptureBeforeCompact . . . 14
- 13.2 SkipCaptureBeforeCompact . . . 15
- 13.3 CompactContext . . . 15

14 Session End **16**

15 System Invariants **16**

16 Hook Event Map **17**

17 Implementation Validation **17**
17.1 Hook Wiring Status 17
17.2 Not Yet Built 18
17.3 Coverage Summary 18

1 Introduction

This specification extends the base Claude Code state machine (`claude-code.tex`) with quarry automatic knowledge capture state. Quarry provides local semantic search over codebases, documents, and conversation history.

The key properties verified by this model:

1. **Knowledge before work**: the codebase is indexed before Claude begins processing. The SessionStart hook auto-registers and syncs the working directory.
2. **No knowledge loss on compaction**: before context compaction shrinks the conversation, the `PreCompact` hook captures session notes. No path reaches `CompactContext` without the capture flag being set.
3. **WebFetch auto-capture**: URLs fetched during a session are automatically ingested into the `web-captures` collection.
4. **Capture toggles are respected**: each auto-capture layer (session sync, web fetch, compaction) can be independently disabled via `.claude/quarry.local.md`.

2 Base Types (from claude-code.tex)

$[TOOLNAME, TOOLINPUT, TOOLOUTPUT, PROMPT,$
$RESPONSE, AGENTID, SESSIONID, CONTEXTCHUNK]$

$SessionPhase ::= spInactive \mid spIdle \mid spProcessing \mid spResponding \mid spEnded$

$ToolPhase ::= tpNone \mid tpPreHook \mid tpPermission \mid tpExecuting \mid tpPostHook$

$PermissionMode ::= pmDefault \mid pmPlan \mid pmAcceptEdits \mid pmDontAsk \mid pmBypass$

$PermissionDecision ::= pdAllow \mid pdDeny \mid pdAsk$

$HookDecision ::= hdAllow \mid hdBlock$

$SessionStartSource ::= ssStartup \mid ssResume \mid ssClear \mid ssCompact$

$SessionEndReason ::= seClear \mid seLogout \mid sePromptExit \mid seBypassDisabled \mid seOther$

$ZBOOL ::= ztrue \mid zfalse$

3 Quarry Types

$[DBNAME, URL]$

`DBNAME` identifies a quarry database (e.g., `default`, `punt-labs`). `URL` identifies a fetched web resource.

4 Global Constants

$$
\begin{array}{l}
maxTools : \mathbb{N} \\
maxSubagents : \mathbb{N} \\
maxContext : \mathbb{N} \\
maxCaptures : \mathbb{N} \\
\hline
maxTools = 5 \\
maxSubagents = 2 \\
maxContext = 3 \\
maxCaptures = 5
\end{array}
$$

5 State

State

$sessionId : \mathbb{F}\ SESSIONID$
$sessionPhase : SessionPhase$
$permMode : PermissionMode$
$toolPhase : ToolPhase$
$currentTool : \mathbb{F}\ TOOLNAME$
$toolsThisTurn : \mathbb{N}$
$activeSubagents : \mathbb{F}\ AGENTID$
$context : \mathrm{seq}\ CONTEXTCHUNK$
$pendingPrompt : \mathbb{F}\ PROMPT$
$pendingResponse : \mathbb{F}\ RESPONSE$
$codebaseIndexed : ZBOOL$
$activeDb : \mathbb{F}\ DBNAME$
$capturedUrls : \mathbb{F}\ URL$
$compactionSaved : ZBOOL$
$cfgSessionSync : ZBOOL$
$cfgWebFetch : ZBOOL$
$cfgCompaction : ZBOOL$

$\#sessionId \leq 1$
$sessionPhase = spInactive \Rightarrow sessionId = \varnothing$
$sessionPhase \neq spInactive \Rightarrow \#sessionId = 1$
$sessionPhase \neq spProcessing \Rightarrow toolPhase = tpNone$
$\#currentTool \leq 1$
$toolPhase = tpNone \Rightarrow currentTool = \varnothing$
$toolPhase \neq tpNone \Rightarrow \#currentTool = 1$
$toolsThisTurn \leq maxTools$
$sessionPhase = spInactive \Rightarrow activeSubagents = \varnothing$
$sessionPhase = spEnded \Rightarrow activeSubagents = \varnothing$
$\#activeSubagents \leq maxSubagents$
$\#context \leq maxContext$
$\#pendingPrompt \leq 1$
$sessionPhase = spIdle \Rightarrow pendingPrompt = \varnothing$
$sessionPhase = spInactive \Rightarrow pendingPrompt = \varnothing$
$sessionPhase = spEnded \Rightarrow pendingPrompt = \varnothing$
$\#pendingResponse \leq 1$
$sessionPhase \neq spResponding \Rightarrow pendingResponse = \varnothing$
$\#activeDb \leq 1$
$\#capturedUrls \leq maxCaptures$
$sessionPhase = spInactive \Rightarrow codebaseIndexed = zfalse$
$sessionPhase = spInactive \Rightarrow activeDb = \varnothing$
$sessionPhase = spInactive \Rightarrow capturedUrls = \varnothing$
$sessionPhase = spInactive \Rightarrow compactionSaved = zfalse$

6 Initialization

Init

$State'$

$sessionId' = \varnothing$
$sessionPhase' = spInactive$
$permMode' = pmDefault$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = 0$
$activeSubagents' = \varnothing$
$context' = \langle\rangle$
$pendingPrompt' = \varnothing$
$pendingResponse' = \varnothing$
$codebaseIndexed' = zfalse$
$activeDb' = \varnothing$
$capturedUrls' = \varnothing$
$compactionSaved' = zfalse$
$cfgSessionSync' = ztrue$
$cfgWebFetch' = ztrue$
$cfgCompaction' = ztrue$

7 Quarry Frame Condition

QuarryFrame

$\Delta State$

$codebaseIndexed' = codebaseIndexed$
$activeDb' = activeDb$
$capturedUrls' = capturedUrls$
$compactionSaved' = compactionSaved$
$cfgSessionSync' = cfgSessionSync$
$cfgWebFetch' = cfgWebFetch$
$cfgCompaction' = cfgCompaction$

8 Session Lifecycle Operations

8.1 StartSessionWithSync

`SessionStart` hook fires. If `cfgSessionSync = ztrue`, quarry auto-registers the working directory and syncs. The codebase is indexed before Claude can process prompts.

$StartSessionWithSync$

$\Delta State$
$sid? : SESSIONID$
$source? : SessionStartSource$
$mode? : PermissionMode$

$sessionPhase = spInactive$
$cfgSessionSync = ztrue$
$sessionPhase' = spIdle$
$sessionId' = \{sid?\}$
$permMode' = mode?$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = 0$
$activeSubagents' = \varnothing$
$context' = \langle\rangle$
$pendingPrompt' = \varnothing$
$pendingResponse' = \varnothing$
$codebaseIndexed' = ztrue$
$activeDb' = \varnothing$
$capturedUrls' = \varnothing$
$compactionSaved' = zfalse$
$cfgSessionSync' = cfgSessionSync$
$cfgWebFetch' = cfgWebFetch$
$cfgCompaction' = cfgCompaction$

8.2 StartSessionNoSync

Session starts but sync is disabled. The codebase is not indexed automatically.

$StartSessionNoSync$

$\Delta State$
$sid? : SESSIONID$
$source? : SessionStartSource$
$mode? : PermissionMode$

$sessionPhase = spInactive$
$cfgSessionSync = zfalse$
$sessionPhase' = spIdle$
$sessionId' = \{sid?\}$
$permMode' = mode?$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = 0$
$activeSubagents' = \varnothing$
$context' = \langle\rangle$
$pendingPrompt' = \varnothing$
$pendingResponse' = \varnothing$
$codebaseIndexed' = zfalse$
$activeDb' = \varnothing$
$capturedUrls' = \varnothing$
$compactionSaved' = zfalse$
$cfgSessionSync' = cfgSessionSync$
$cfgWebFetch' = cfgWebFetch$
$cfgCompaction' = cfgCompaction$

8.3 SubmitPrompt

SubmitPrompt

$QuarryFrame$
$prompt? : PROMPT$
$hookResult? : HookDecision$
$contextChunk? : CONTEXTCHUNK$

$sessionPhase = spIdle$
$hookResult? = hdAllow$
$sessionPhase' = spProcessing$
$pendingPrompt' = \{prompt?\}$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle contextChunk? \rangle)$
$toolsThisTurn' = 0$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$sessionId' = sessionId$
$permMode' = permMode$
$activeSubagents' = activeSubagents$
$pendingResponse' = \varnothing$

RejectPrompt

$\Xi State$
$prompt? : PROMPT$
$hookResult? : HookDecision$

$sessionPhase = spIdle$
$hookResult? = hdBlock$

9 Tool Call Operations

9.1 BeginToolCall

BeginToolCall

$QuarryFrame$
$tool? : TOOLNAME$

$sessionPhase = spProcessing$
$toolPhase = tpNone$
$toolsThisTurn < maxTools$
$toolPhase' = tpPreHook$
$currentTool' = \{tool?\}$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$context' = context$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

9.2 PreToolHookAllow

PreToolHookAllow

$QuarryFrame$
$hookDecision? : PermissionDecision$
$needsPermission? : ZBOOL$

$sessionPhase = spProcessing$
$toolPhase = tpPreHook$
$hookDecision? = pdAllow$
$toolPhase' = \mathbf{if}\ needsPermission? = ztrue\ \mathbf{then}\ tpPermission\ \mathbf{else}\ tpExecuting$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$currentTool' = currentTool$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$context' = context$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

9.3 PreToolHookDeny

PreToolHookDeny

$QuarryFrame$
$hookDecision? : PermissionDecision$
$denialContext? : CONTEXTCHUNK$

$sessionPhase = spProcessing$
$toolPhase = tpPreHook$
$hookDecision? = pdDeny$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle denialContext? \rangle)$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

9.4 GrantPermission

GrantPermission

$QuarryFrame$

$sessionPhase = spProcessing$
$toolPhase = tpPermission$
$toolPhase' = tpExecuting$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$currentTool' = currentTool$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$context' = context$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

9.5 DenyPermission

DenyPermission

$QuarryFrame$
$denialContext? : CONTEXTCHUNK$

$sessionPhase = spProcessing$
$toolPhase = tpPermission$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle denialContext? \rangle)$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

9.6 CompleteToolSuccess

CompleteToolSuccess

$QuarryFrame$
$output? : CONTEXTCHUNK$

$sessionPhase = spProcessing$
$toolPhase = tpExecuting$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = toolsThisTurn + 1$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle output? \rangle)$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$activeSubagents' = activeSubagents$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

9.7 CompleteToolFailure

CompleteToolFailure

$QuarryFrame$
$errorContext? : CONTEXTCHUNK$

$sessionPhase = spProcessing$
$toolPhase = tpExecuting$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = toolsThisTurn + 1$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle errorContext? \rangle)$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$activeSubagents' = activeSubagents$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

10 Quarry Operations

10.1 CaptureWebFetch

`PostToolUse` on `WebFetch`. When web fetch capture is enabled, the fetched URL is added to the captured set and ingested into the `web-captures` collection (async, fire-and-forget).

$$
\begin{array}{|l}
\hline
\textit{CaptureWebFetch} \\
\Delta State \\
url? : URL \\
\hline
sessionPhase = spProcessing \\
toolPhase = tpNone \\
cfgWebFetch = ztrue \\
url? \notin capturedUrls \\
\#capturedUrls < maxCaptures \\
capturedUrls' = capturedUrls \cup \{url?\} \\
sessionPhase' = sessionPhase \\
sessionId' = sessionId \\
permMode' = permMode \\
toolPhase' = toolPhase \\
currentTool' = currentTool \\
toolsThisTurn' = toolsThisTurn \\
activeSubagents' = activeSubagents \\
context' = context \\
pendingPrompt' = pendingPrompt \\
pendingResponse' = pendingResponse \\
codebaseIndexed' = codebaseIndexed \\
activeDb' = activeDb \\
compactionSaved' = compactionSaved \\
cfgSessionSync' = cfgSessionSync \\
cfgWebFetch' = cfgWebFetch \\
cfgCompaction' = cfgCompaction \\
\hline
\end{array}
$$

10.2 SwitchDatabase

Claude calls `quarry use <name>`. Changes the active database for subsequent queries.

SwitchDatabase

$\Delta State$
$db? : DBNAME$

$sessionPhase = spProcessing$
$toolPhase = tpNone$
$activeDb' = \{db?\}$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = toolPhase$
$currentTool' = currentTool$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$context' = context$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$
$codebaseIndexed' = codebaseIndexed$
$capturedUrls' = capturedUrls$
$compactionSaved' = compactionSaved$
$cfgSessionSync' = cfgSessionSync$
$cfgWebFetch' = cfgWebFetch$
$cfgCompaction' = cfgCompaction$

10.3 SearchKnowledge

Claude calls `quarry find`. This is a read-only query that injects search results into context. Modeled to show that search results flow into the conversation.

SearchKnowledge

$\Delta State$
$results? : CONTEXTCHUNK$

$sessionPhase = spProcessing$
$toolPhase = tpNone$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle results? \rangle)$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = toolPhase$
$currentTool' = currentTool$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$
$codebaseIndexed' = codebaseIndexed$
$activeDb' = activeDb$
$capturedUrls' = capturedUrls$
$compactionSaved' = compactionSaved$
$cfgSessionSync' = cfgSessionSync$
$cfgWebFetch' = cfgWebFetch$
$cfgCompaction' = cfgCompaction$

10.4 InjectKnowledgeHint

Quarry proactively injects relevant knowledge into context at key moments: `SessionStart` (project standards, workflow tips), `PostToolUse` on `Edit|Write` (e.g., "remember our changelog standard", "update DESIGN.md with an ADR"), `PostToolUse` on tool-specific patterns (e.g., "top hints for working with Z specs" when editing `.tex` files).

These are contextual hints retrieved from quarry collections (standards, design docs, project conventions) and injected as `additionalContext`. The injection is non-blocking and additive — it appends to context without altering other state.

$$
\begin{array}{l}
\textit{InjectKnowledgeHint} \\
\hline
\Delta State \\
hint? : CONTEXTCHUNK \\
\hline
sessionPhase \in \{spIdle, spProcessing\} \\
context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle hint? \rangle) \\
sessionPhase' = sessionPhase \\
sessionId' = sessionId \\
permMode' = permMode \\
toolPhase' = toolPhase \\
currentTool' = currentTool \\
toolsThisTurn' = toolsThisTurn \\
activeSubagents' = activeSubagents \\
pendingPrompt' = pendingPrompt \\
pendingResponse' = pendingResponse \\
codebaseIndexed' = codebaseIndexed \\
activeDb' = activeDb \\
capturedUrls' = capturedUrls \\
compactionSaved' = compactionSaved \\
cfgSessionSync' = cfgSessionSync \\
cfgWebFetch' = cfgWebFetch \\
cfgCompaction' = cfgCompaction
\end{array}
$$

This operation fires in both `spIdle` (SessionStart context injection) and `spProcessing` (PostToolUse context injection). The hint content is determined by quarry's semantic search over registered collections — the Z spec abstracts this as an opaque `CONTEXTCHUNK` input.

Example injection points:

- `SessionStart`: "This project follows punt-kit standards. Key conventions: ruff, mypy, pytest, CHANGELOG.md under [Unreleased]."
- `PostToolUse` on `Edit` of `CHANGELOG.md`: "Keep a Changelog format. Entries under [Unreleased] with Added/Changed/Fixed sections."
- `PostToolUse` on `Edit` of `*.tex`: "Z spec conventions: use \quad~ for indentation, ZBOOL for booleans, bounded sets for ProB animation."
- `PostToolUse` on `Bash` with git commands: "Session close protocol: git status, git add, bd sync, git commit, git push."

11 Subagent Operations

SpawnSubagent

$QuarryFrame$
$agentId? : AGENTID$

$sessionPhase = spProcessing$
$agentId? \notin activeSubagents$
$\#activeSubagents < maxSubagents$
$activeSubagents' = activeSubagents \cup \{agentId?\}$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = toolPhase$
$currentTool' = currentTool$
$toolsThisTurn' = toolsThisTurn$
$context' = context$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

StopSubagent

$QuarryFrame$
$agentId? : AGENTID$
$hookResult? : HookDecision$
$resultContext? : CONTEXTCHUNK$

$sessionPhase = spProcessing$
$agentId? \in activeSubagents$
$hookResult? = hdAllow$
$activeSubagents' = activeSubagents \setminus \{agentId?\}$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle resultContext? \rangle)$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = toolPhase$
$currentTool' = currentTool$
$toolsThisTurn' = toolsThisTurn$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$

12 Response and Stop

BeginResponse

$QuarryFrame$
$response? : RESPONSE$

$sessionPhase = spProcessing$
$toolPhase = tpNone$
$activeSubagents = \varnothing$
$sessionPhase' = spResponding$
$pendingResponse' = \{response?\}$
$pendingPrompt' = \varnothing$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$context' = context$

FinishResponse

$QuarryFrame$
$hookResult? : HookDecision$
$responseContext? : CONTEXTCHUNK$

$sessionPhase = spResponding$
$hookResult? = hdAllow$
$sessionPhase' = spIdle$
$pendingResponse' = \varnothing$
$context' = (1 \mathinner{..} maxContext) \lhd (context \frown \langle responseContext? \rangle)$
$toolsThisTurn' = 0$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$activeSubagents' = activeSubagents$
$pendingPrompt' = \varnothing$

ForceContinue

$QuarryFrame$
$hookResult? : HookDecision$

$sessionPhase = spResponding$
$hookResult? = hdBlock$
$sessionPhase' = spProcessing$
$pendingResponse' = \varnothing$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$context' = context$
$pendingPrompt' = pendingPrompt$

13 Context Management — Compaction Safety

This is the critical section for quarry. Context compaction destroys conversation history. The `PreCompact` hook must capture session notes *before* the context is shrunk.

13.1 CaptureBeforeCompact

`PreCompact` hook fires. If compaction capture is enabled, quarry reads the transcript and ingests it as session notes into the `session-notes` collection. This must happen before `CompactContext`.

$$
\begin{array}{l}
\textit{CaptureBeforeCompact} \\
\hline
\Delta State \\
\hline
sessionPhase = spIdle \\
cfgCompaction = ztrue \\
\#context > 0 \\
compactionSaved' = ztrue \\
sessionPhase' = sessionPhase \\
sessionId' = sessionId \\
permMode' = permMode \\
toolPhase' = toolPhase \\
currentTool' = currentTool \\
toolsThisTurn' = toolsThisTurn \\
activeSubagents' = activeSubagents \\
context' = context \\
pendingPrompt' = pendingPrompt \\
pendingResponse' = pendingResponse \\
codebaseIndexed' = codebaseIndexed \\
activeDb' = activeDb \\
capturedUrls' = capturedUrls \\
cfgSessionSync' = cfgSessionSync \\
cfgWebFetch' = cfgWebFetch \\
cfgCompaction' = cfgCompaction
\end{array}
$$

13.2 SkipCaptureBeforeCompact

Compaction capture is disabled. The `PreCompact` hook is a no-op. Session notes are not saved.

$$
\begin{array}{l}
\textit{SkipCaptureBeforeCompact} \\
\hline
\Xi State \\
\hline
sessionPhase = spIdle \\
cfgCompaction = zfalse
\end{array}
$$

13.3 CompactContext

Context is shrunk. The `compactionSaved` flag records whether notes were captured. After compaction it resets to `zfalse` for the next potential compaction cycle.

CompactContext

$\Delta State$
$newLen? : \mathbb{N}$

$sessionPhase = spIdle$
$\#context > 0$
$newLen? < \#context$
$newLen? \leq maxContext$
$context' = (1 \mathinner{..} newLen?) \lhd context$
$compactionSaved' = zfalse$
$sessionPhase' = sessionPhase$
$sessionId' = sessionId$
$permMode' = permMode$
$toolPhase' = toolPhase$
$currentTool' = currentTool$
$toolsThisTurn' = toolsThisTurn$
$activeSubagents' = activeSubagents$
$pendingPrompt' = pendingPrompt$
$pendingResponse' = pendingResponse$
$codebaseIndexed' = codebaseIndexed$
$activeDb' = activeDb$
$capturedUrls' = capturedUrls$
$cfgSessionSync' = cfgSessionSync$
$cfgWebFetch' = cfgWebFetch$
$cfgCompaction' = cfgCompaction$

14 Session End

EndSession

$\Delta State$
$reason? : SessionEndReason$

$sessionPhase = spIdle$
$activeSubagents = \varnothing$
$sessionPhase' = spEnded$
$sessionId' = \varnothing$
$toolPhase' = tpNone$
$currentTool' = \varnothing$
$toolsThisTurn' = 0$
$activeSubagents' = \varnothing$
$context' = \langle\rangle$
$pendingPrompt' = \varnothing$
$pendingResponse' = \varnothing$
$permMode' = permMode$
$codebaseIndexed' = zfalse$
$activeDb' = \varnothing$
$capturedUrls' = \varnothing$
$compactionSaved' = zfalse$
$cfgSessionSync' = cfgSessionSync$
$cfgWebFetch' = cfgWebFetch$
$cfgCompaction' = cfgCompaction$

15 System Invariants

1. **Codebase indexed when sync enabled**: if `cfgSessionSync = ztrue` and the session started via `StartSessionWithSync`, then `codebaseIndexed = ztrue` for the duration of the session.

2. **Capture config is immutable during session**: the `cfg*` flags are set at `Init` and preserved by all operations. They cannot change mid-session.

3. **WebFetch deduplication**: `CaptureWebFetch` requires `url?` $\notin$ `capturedUrls`. No URL is ingested twice in the same session.

4. **Active database consistency**: at most one database is active. `SwitchDatabase` replaces, does not accumulate.

5. **Compaction flag lifecycle**: CaptureBeforeCompact sets compactionSaved to ztrue. CompactContext resets it to zfalse. This tracks whether the most recent compaction was preceded by a capture.

6. **Clean session boundaries**: all quarry state (indexed, activeDb, captures, compaction flag) is cleared on `EndSession`.

Note on the compaction safety property: The Z spec does *not* enforce that capture must precede compaction — it is possible to compact without capturing (the operations are independent). This reflects reality: Claude Code fires the pre-compact hook before compaction, but the hook is non-blocking. The compactionSaved flag lets us *observe* whether the capture occurred, not *enforce* it. Enforcement is the hook system's responsibility.

16 Hook Event Map

Hook	Matcher	Operation	Effect	Blk
SessionStart	*	StartSession-WithSync/NoSync	Auto-register + sync	No
SessionStart	*	InjectKnowledgeHint	Project standards	No
PostToolUse	WebFetch	CaptureWebFetch	Ingest URL	No
PostToolUse	Edit—Write	InjectKnowledgeHint	File-type hints	No
PostToolUse	Bash	InjectKnowledgeHint	Workflow hints	No
PostToolUse	quarry__use	SwitchDatabase	Change active DB	No
PostToolUse	quarry__find	SearchKnowledge	Inject results	No
PostToolUse	quarry__*	(suppress output)	Format display	No
PreCompact	*	CaptureBefore-Compact	Save session notes	No
SessionEnd	*	EndSession	Clear state	No

17 Implementation Validation

This section cross-references the Z specification against the current quarry implementation: the hook config, shell scripts in the hooks directory, and the Python handlers.

17.1 Hook Wiring Status

All three knowledge capture handlers are implemented in `src/quarry/hooks.py` and wired via shell scripts in `hooks/hooks.json`.

Component	Hook Event	Files	Status
Auto-register + sync	SessionStart	session-start.sh, session-sync.sh, hooks.py	Correct
Output formatting	PostToolUse (quarry)	suppress-output.sh	Correct
Web fetch capture	PostToolUse (WebFetch)	web-fetch.sh, hooks.py	Correct
Compaction capture	PreCompact	pre-compact.sh, hooks.py	Correct

17.2 Not Yet Built

Z Operation	Hook Event	Effect	Effort
InjectKnowledgeHint	SessionStart	Project standards	Medium
InjectKnowledgeHint	PostToolUse Edit—Write	File-type hints	Medium
InjectKnowledgeHint	PostToolUse Bash	Workflow hints	Medium
EndSession cleanup	SessionEnd	Clear session state	Low

Knowledge hint injection is the most significant unbuilt capability. It requires a hint resolution engine that maps the current activity (tool name, file path, command) to relevant quarry collections, plus careful scoping: hints must be short (1–3 sentences) to avoid flooding context.

An earlier attempt at convention hints via a PreToolUse hook accumulator (see ADR in `DESIGN.md`) was implemented and later removed (PR #113) in favor of a future semantic approach.

17.3 Coverage Summary

Z Operation	Implementation	Status
StartSessionWithSync	hooks.py + session-sync.sh	Correct
StartSessionNoSync	hooks.py (config check)	Correct
SubmitPrompt / RejectPrompt	(base)	Correct
Tool call operations	(base)	Correct
CaptureWebFetch	hooks.py + web-fetch.sh	Correct
SwitchDatabase	MCP tool	Correct
SearchKnowledge	MCP tool	Correct
InjectKnowledgeHint	—	**Not built**
CaptureBeforeCompact	hooks.py + pre-compact.sh	Correct
CompactContext	(base)	Correct
EndSession	—	**Not built**